
Problem 1: Algorithm Properties and Correctness

a) Evaluate both algorithms against the five fundamental properties

- **Algorithm A:**
    1. **Input:** Satisfied. Takes temperature array and n as inputs.
    2. **Output:** Satisfied. Returns a single value average.
    3. **Definiteness:** Satisfied. Every step (initialization, addition, division) is clearly defined and unambiguous.
    4. **Finiteness:** Satisfied. The loop runs exactly n times and terminates.
    5. **Effectiveness:** Satisfied. All operations (addition, division) are basic and executable.
    - **Conclusion:** Algorithm A satisfies all properties.
- **Algorithm B:**
    1. **Input:** Satisfied.
    2. **Output:** Satisfied. Returns sum.
    3. **Definiteness:** Satisfied. Steps are clear.
    4. **Finiteness:** Satisfied. Both loops run for a fixed number of times (n).
    5. **Effectiveness:** Satisfied. Operations are executable.
    - **Conclusion:** Algorithm B satisfies the formal properties of an algorithm, even though the logic produces an incorrect result (it is a valid algorithm, just a logically flawed one).

b) Step-by-step execution

Input: temperatures = [20, 25, 22, 24, 21], n = 5

Trace: Algorithm A

1. total = 0
2. Loop i = 1 to 5:
    - i=1: total = 0 + 20 = 20
    - i=2: total = 20 + 25 = 45
    - i=3: total = 45 + 22 = 67
    - i=4: total = 67 + 24 = 91

    - i=5: total = 91 + 21 = 112
3. average = 112 / 5 = **22.4**
4. Return 22.4

- **Result:** Correct.

Trace: Algorithm B

1. sum = temperature[1] = 20
2. Loop i = 2 to 5 (Summation):
    - sum = 20 + 25 = 45
    - sum = 45 + 22 = 67
    - sum = 67 + 24 = 91
    - sum = 91 + 21 = 112
3. Loop i = 1 to 5 (Division):
    - i=1: sum = 112 / 5 = 22.4
    - i=2: sum = 22.4 / 5 = 4.48
    - i=3: sum = 4.48 / 5 = 0.896
    - i=4: sum = 0.896 / 5 = 0.1792
    - i=5: sum = 0.1792 / 5 = 0.03584
4. Return **0.03584**

- **Result:** Incorrect.

c) logical Error in Algorithm B

- **Identification:** The error is in lines 4–5. The algorithm places the division operation sum = sum / n inside a loop that iterates n times.
- **Explanation:** Instead of dividing the total sum by n once, it divides the sum by n repeatedly (n times).
- **Principle Violated:** The definition of **Arithmetic Mean**. The mean is the sum of terms divided by the count of terms *exactly once*.

d) Comparison and Selection

- **Simplicity:** Algorithm A is simpler and more readable. Algorithm B essentially has two separate loops, making it unnecessarily complex.
- **Correctness:** Algorithm A is correct. Algorithm B is incorrect.

- **Efficiency:** Both are $O(n)$, but Algorithm A does it in one pass (1 loop), while Algorithm B uses two passes (2 loops), making Algorithm A slightly more efficient in practice.
- **Choice: Algorithm A** should be chosen because it produces the correct mathematical result and is cleaner to maintain.

e) Test Strategy

1. **Standard Case:** [10, 20, 30], n=3. Expected: 20.
2. **Single Element (n=1):** [15], n=1. Expected: 15. (Tests if loops handle single iterations).
3. **Empty/Zero (n=0):** [], n=0. Expected: Error handling or 0. (Tests division by zero risk).
4. **Negative Numbers:** [-5, -10, 5], n=3. Expected: -3.33. (Tests signed integer handling).
5. **Large Numbers:** [1000000, 2000000], n=2. Expected: 1500000. (Tests for variable overflow).

Problem 2: Time Complexity Analysis and Comparison

a) Exact number of operations (Worst Case for n = 10,000,000)

- **Algorithm A (Linear Search):** In the worst case (user not found or at the end), we check every user.
    - **Operations:** 10,000,000 comparisons.
- **Algorithm B (Binary Search):** The number of comparisons is log_2(n) + 1.
    - $\log_2(10,000,000) ~ 23.25.
    - **Operations:** ~24 comparisons.
- **Algorithm C (Hash Table):** A good hash table has constant time lookup on average.
    - **Operations:** 1 comparison (assuming no collisions).

b) Time Complexity (Big-O)

- **Algorithm A:** $O(n)$.
    - *Justification:* The number of operations grows linearly with the input size n. If n doubles, operations double.
- **Algorithm B:** O(log n).

- *Justification:* The search space is divided in half with every iteration. The number of steps required to reach 1 is the logarithm of n base 2.

- **Algorithm C:** O(1) (Average case).
  - *Justification:* The time to compute the hash and index into the array does not depend on the size of the dataset (n).

c) Comparison Table

n (Users)	Algorithm A (Linear)	Algorithm B (Binary)	Algorithm C (Hash)
100	100	~7	1
1,000	1,000	~10	1
10,000	10,000	~14	1
100,000	100,000	~17	1
1,000,000	1,000,000	~20	1
10,000,000	10,000,000	~24	1

- **Pattern:** Linear search explodes as n grows. Binary search grows very slowly. Hash lookup remains constant.

d) Throughput Analysis

Load: 50,000 checks/sec.

Cost per op: 1 nanosecond (10^{-9} sec).

1. **Algorithm A:**
   - Ops per check: 10^7
   - Total ops/sec: $50,000 \times 10^7 = 5 \times 10^{11} ops.
   - Time required: $5 \times 10^{11} \times 10^{-9} = 500 seconds.
   - **Result:** Cannot handle load (Requires 500s to process 1s of traffic).
2. **Algorithm B:**
   - Ops per check: ~24

    - Total ops/sec: $50,000 \times 24 = 1.2 \times 10^6 ops.
    - Time required: 1.2 \times 10^6 \times 10^{-9} = 0.0012 seconds.
    - **Result:** Can handle load easily.
3. **Algorithm C:**
    - Ops per check: 1
    - Time required: 0.00005 seconds.
    - **Result:** Can handle load easily.

e) Sorting Trade-off

Let k be the number of searches.

- Cost of **Linear Search**: k.n
- Cost of **Sort + Binary Search**: (n log n) + (k .log n)

We need: (n log n) + (k log n) < k . n

Solving for k:

n log n < k(n - log n)

k > {n log n}{n - log n}

Since log n is very small compared to n, this approximates to k > log n.

- **Condition:** It is worthwhile if you perform more than log n searches (roughly 24 searches for 10M users).

Problem 3: Space Complexity and Space-Time Tradeoffs

a) Time Complexity Analysis

- **Implementation A (Nested Loop):**
    - For every prerequisite (outer loop runs m times), we iterate through all completed courses (inner loop runs n times).
    - **Complexity:** O(m x n)
- **Implementation B (Hash Set):**
    - Creating the set: Iterating through completed courses takes O(n).
    - Checking prerequisites: Iterating through prerequisites is O(m), and set lookup is O(1). Total O(m).
    - **Complexity:** O(n + m)

b) Space Complexity Analysis

- **Implementation A:** Uses existing arrays. No extra data structures created.
    - **Complexity:** O(1) auxiliary space.
- **Implementation B:** Creates a Hash Set storing all n completed courses.
    - **Complexity:** O(n) auxiliary space.

c) Operational Calculations (Daily)

Given: n=40, m=5, Students = 100,000.

1. **Total Comparisons for Implementation A:**
    - Worst case per student: m x n = 5 x 40 = 200 comparisons.
    - Total per day: 100,000 x 200 = 20,000,000$ (20 Million) comparisons.
2. **Total Memory for Implementation B:**
    - Memory per student request: n x 8 bytes = 40 x 8 = 320 bytes.
    - Note: Memory is usually released after the request, so "Total Memory per day" refers to allocation churn, but if calculating concurrent peak, see section (d). Total allocation volume = 100,000 x 320 bytes = 32 MB total allocated over the day.

d) Constraint Analysis (10,000 Concurrent Requests)

- **Constraint 1:** Response time < 10ms (10,000 \mu s).
    - **Imp A:** 200 comparisons x 0.1 \mu s = 20 \mu s. (Passes).
    - **Imp B:** (40 + 5) operations x 0.1 \mu s = 4.5 \mu s. (Passes).
- **Constraint 2:** Memory < 1GB for 10,000 concurrent users.
    - **Imp A:** 0 extra memory. Total = 0. (Passes).
    - **Imp B:** 320 bytes per user x 10,000 = 3,200,000 bytes = 3.2 MB. (Passes).

Justification: Both pass, but **Implementation B** is theoretically faster (O(n+m) vs O(nm)). However, given n and m are very small (40 and 5), the O(n) overhead to build the hash set might actually be slower than the simple loop in practice due to memory allocation overhead.

- *Decision:* For this specific constraint (n=40), **Implementation A** is likely better because it uses zero memory allocation overhead, and 200 operations is trivial for a CPU. If n were large (e.g., 10,000), B would be required.

e) Hybrid Approach

- **Strategy:** Check the size of completedCourses (n) and prerequisites (m).
- **Threshold:**
    - If $m \times n < C (where C is a constant representing the overhead of creating a HashSet, roughly 50-100 ops), use **Nested Loops (Imp A)** to save memory allocation time.
    - If m x n is large (e.g., > 100), use **Hash Set (Imp B)**.
- **Logic:** For very small lists (1-2 prerequisites), the cost of new HashSet() is higher than just doing 5-10 comparisons.

Problem 4: Asymptotic Notation Applications

a) Formal Proof of Worst Case

Definition of Big-O: T(n) is O(g(n)) if there exist positive constants c and n_0 such that T(n) < c . g(n) for all n > n_0.

1. **Bubble Sort (O(n^2)):**
    - The algorithm performs (n-1) + (n-2) + ... + 1 comparisons.
    - T(n) = {n(n-1)}/{2} = {n^2}/{2} - {n}/{2}$.
    - We need {n^2}/{2} - {n}/{2} < c . n^2.
    - Let c = 1. Since {n^2}/{2} < n^2 for all n > 1, the condition holds. Thus, T(n) O(n^2).
2. **Merge Sort (O(n \log n)):**
    - Recurrence relation: T(n) = 2T(n/2) + n.
    - Using Master Theorem or substitution, this resolves to n log_2 n.
    - We need n log_2 n < c . (n log n).
    - Let c = 1, n_0 = 1. The inequality holds. Thus, T(n) belongs O(n \log n).

b) Algorithm Selection by Data Type

- **Type A (Nearly Sorted): Insertion Sort**.
    - *Justification:* Insertion sort has a best-case complexity of O(n) when data is sorted. For nearly sorted data, it runs close to O(n), making it extremely fast. Merge/Quick sort would still do O(n log n).
- **Type B (Random): Quick Sort**.

    - *Justification:* While worst-case is O(n^2), Quick Sort typically outperforms Merge Sort on random data due to smaller constant factors and better cache locality (in-place sorting).

- **Type C (Reverse Sorted): Merge Sort**.
    - *Justification:* This is the worst-case for Insertion Sort (O(n^2)) and potentially Quick Sort (depending on pivot implementation). Merge Sort guarantees Theta(n log n) regardless of order.

c) Θ vs O Notation

- **Merge Sort (Theta(n log n)):** The symbol Theta (Theta) means "Tight Bound". Merge sort takes n log n steps in the Best case AND the Worst case. It never runs faster or slower than this rate.
- **Quick Sort (O(n log n) average):** The symbol O (Big-O) sets an upper bound. In the average case, Quick Sort is upper-bounded by n log n. However, because its worst case is O(n^2) and best case is O(n log n), we cannot use Θ to describe all cases universally without specifying the case.

d) Decision Tree

1. **Check n (Data Size):**
    - **If Small (n < 50):** Use **Insertion Sort**. (Low overhead, fast for small inputs).
    - **If Medium/Large (n > 50):** Check Data Characteristics.
        - **If Nearly Sorted:** Use **Insertion Sort**.
        - **If Random:** Use **Quick Sort** (Standard library implementations usually use this).
        - **If Reverse/Unknown/Critical Limit:** Use **Merge Sort** (Guaranteed performance safety).

e) Mystery Sort Analysis

- **Mystery Sort:** T_1(n) = 5n^2 + 100n + 1000
- **Merge Sort:** T_2(n) = 10n . log_2 n

We are looking for values where T_1(n) < T_2(n).

Comparing growth rates: n^2 grows much faster than n log n.

- At n=1: Mystery=1105, Merge=0.
- At n=10: Mystery=2500, Merge=332.

- At n=100: Mystery=50000+, Merge=6600.

Mathematical Conclusion: Since the quadratic term 5n^2 dominates 10n log n immediately and the constant term (1000) is large, **Mystery Sort is never faster than Merge Sort** for any positive integer n. The proposed algorithm is inefficient compared to Merge Sort for all dataset sizes.